

Gib deiner KI ein Gedächtnis, das die Sitzung überlebt.

Deine Coding-Tools vergessen zwischen zwei Sitzungen alles. Diese Anleitung verbindet sie mit einem Brain, das sich merkt, was du entschieden hast und warum — und es dir zurückgibt, bevor du es erneut erklären musst. Drei Schritte.

In **5 Minuten** einsatzbereit

10 Clients unterstützt

Ein Befehl

1

Token

Token erstellen

Melde dich an, öffne **Settings** → **Tokens** und klicke auf **Create token**.

Benenne ihn nach dem *Rechner*, nicht nach dir — `laptop`, `work-desktop`, `ci-runner`. Widerrufen wird pro Rechner, und genau deshalb machen rechnerbezogene Namen den Widerruf später überhaupt erst nützlich.

Der Token wird nur einmal angezeigt. Gespeichert wird ausschließlich sein SHA-256-Hash — es gibt keine Anzeigen-Schaltfläche und keine Wiederherstellung. Verloren? Erstelle einen neuen und widerrufe den alten.

Danach zeigt die Seite einen Installationsbefehl an, in dem dein Token bereits eingesetzt ist. Dieser Befehl ist Schritt 2 — kopieren und weiterspringen.

2

Verbinden

Verbinde dein KI-Tool

Ein Befehl, unabhängig vom Tool. `--client` ist standardmäßig `claude-code`.

● ● ● MACOS · LINUX · WSL · GIT BASH

```
curl -fsSL https://<your-brain>/api/onboard.sh \  
| bash -s 'bp_...' --client claude-code
```

● ● ● WINDOWS POWERSHELL 5.1+

```
iwr https://<your-brain>/api/onboard.ps1 -UseBasicParsing | iex  
Install-Brain -Token 'bp_...' -Client claude-code
```

WÄHLE DEINEN CLIENT

DEIN TOOL	FLAG	WISSENSWERT
Claude Code	claude-code	Installiert zusätzlich den Brain-Skill
Claude Desktop	claude-desktop	Benötigt Node. Danach die App vollständig beenden — das Fenster zu schließen genügt nicht
Cursor	cursor	
Windsurf	windsurf	
Google Antigravity	antigravity	Eine Konfiguration bedient IDE und CLI gemeinsam
VS Code + Copilot	vscode	Im Projektstammverzeichnis ausführen — schreibt ./.vscode/mcp.json
GitHub Copilot CLI	copilot-cli	
OpenAI Codex	codex	Gibt zusätzlich eine export BRAIN_TOKEN=... -Zeile aus. Ohne sie im Shell-Profil bekommt Codex einen 401
Alles andere	generic	Mit --config-path <Datei> wird diese Datei für dich geschrieben

JetBrains, Visual Studio, Eclipse und Xcode haben keinen festen Konfigurationspfad — füge dort stattdessen das JSON aus der Token-Seite ein.

WAS DIESER BEFEHL TATSÄCHLICH TUT

1. **Schreibt die Konfiguration** — über das eigene `mcp add` deines Tools, sofern vorhanden, sonst durch *Zusammenführen* mit der bestehenden Konfigurationsdatei.

Deine anderen MCP-Server bleiben erhalten, und die Datei wird vorher gesichert.

2. **Beweist, dass es funktioniert** — mit einer echten Verbindung und einem echten Tool-Aufruf über dein Netzwerk und deine Authentifizierung. Nur deshalb darf der Befehl Erfolg melden: dass eine Datei geschrieben wurde, sagt nichts darüber aus, ob der Token überhaupt etwas aufrufen kann.
3. **Protokolliert eine erste Sitzung**, damit dein Dashboard nicht am ersten Tag leer ist.

Starte danach dein KI-Tool neu. Alle lesen die MCP-Konfiguration ausschließlich beim Start. Das ist mit Abstand die häufigste Ursache für „installiert, aber es passiert nichts“.

LIEBER ERST LESEN, BEVOR DU IN EINE SHELL PIPEST?

● ● ● VORHER PRÜFEN

```
curl -fsSL https://<your-brain>/api/onboard.sh -o /tmp/brain-instal
less /tmp/brain-install.sh
bash /tmp/brain-install.sh 'bp_...' --client cursor
```

Sprich mit ihm wie mit einem Kollegen

Du rufst nie Tools beim Namen auf. Du sprichst normal, und der installierte Skill sagt deiner KI, wonach sie greifen soll.

DU SAGST...	WAS PASSIERT
<i>(du beginnst irgendeine echte Aufgabe)</i>	Eine Sitzung startet; passende frühere Regeln gehen an das Modell, bevor es etwas schreibt
<i>„merk dir: wir nutzen pgvector, nicht Pinecone“</i>	Wird als Regel gespeichert und erscheint unter Skills
<i>„frag das Oracle: wie haben wir die Migrationsreihenfolge gelöst?“</i>	Eine zusammengefasste Antwort — mit Belegen zu den verwendeten Regeln
<i>„finde die Regel zu Prisma-Migrationen“</i>	Die inhaltlich nächsten Treffer. Sofort und kostenlos — kein Modellaufruf
<i>„was habe ich letzte Woche an Billing gemacht?“</i>	Frühere Sitzungen, die zu diesen Wörtern passen
<i>„hat funktioniert“ · „wir sind fertig“ · „ab damit“</i>	Schließt die Sitzung — genau dadurch lernt es

DIE EINE GEWOHNHEIT, AUF DIE ES ANKOMMT

Sag „hat funktioniert“, wenn du fertig bist.

Eine nicht geschlossene Sitzung bringt dem Brain nichts bei — die Extraktion läuft erst beim Schließen. Das ist der häufigste Grund, warum ein Brain nach einer Woche echter Nutzung noch leer ist.

Alles andere hier ist optional. Das hier nicht.

EINE SITZUNG VON ANFANG BIS ENDE

● ● ● S0 SIEHT ES AUS

Du Ich muss die Auth-Middleware auf den neuen Credentials-Adapter umbauen.

Claude [öffnet eine Sitzung, holt passende Regeln]
Zwei deiner Regeln greifen hier: Sitzungen binden beim Start, und Credential-Prüfungen gehören in die Repository-Schicht.

Du Nein – wir haben diese Prüfung letzten Monat in den Route-Handler verschoben.

Claude Notiert, wird angepasst.

Du Hat funktioniert, wir sind fertig.

Claude [schließt die Sitzung mit dem Gelernten]
Gespeichert: "Credential-Prüfungen liegen im Route-Handler, nicht in der Repository-Schicht – verschoben 2026-07."

Diese Korrektur ist jetzt dauerhaft. Nächsten Monat, auf einem anderen Rechner, kommt sie an, bevor du dich daran erinnern musst.

WIE DU GUTE ANTWORTEN BEKOMMST

- **„finde die Regel zu X“** — sofort, kostenlos, kein Modellaufruf. Nimm es immer dann, wenn du sonst eine frühere Entscheidung neu herleiten würdest.
- **„frag das Oracle: ...“** — langsamer und kostenpflichtig, dafür zusammengefasst und belegt. Nimm es, wenn du Begründungen willst, keine Liste.
- **Frag das Oracle nicht** nach allgemeinen Programmierfragen. Es ist das Gedächtnis deines Projekts, nicht das Internet. „Wie funktioniert Flexbox“ ist eine Websuche; „wie haben *wir* das Dashboard-Layout entschieden“ ist das Oracle.

✓
Prüfen

Bestätigen, dass es läuft

● ● ● PRÜFEN

```
claude mcp list | grep brain  
# brain: https://.../mcp (HTTP) - ✓ Connected
```

Frag es dann in deinem Tool: „*frag das Brain, was es über dieses Projekt weiß*“. Wenn überhaupt eine Antwort kommt, läuft der Kreislauf.

FALLS ETWAS NICHT STIMMT

SYMPTOM	FAST IMMER
Dein Tool sieht das Brain nicht	Nach der Installation nicht neu gestartet
401 bei jedem Aufruf	Token widerrufen, abgelaufen oder von einem anderen Brain
Nur Codex bekommt 401	BRAIN_TOKEN fehlt im Shell-Profil
Verbunden, aber Skills bleibt leer	Sitzungen werden nie geschlossen — sag „wir sind fertig“
~/.claude/mcp.json bearbeitet, nichts ändert sich	Falsche Datei. Claude Code liest ~/.claude.json

WEITERLESEN

Erste Schritte

Dasselbe, langsamer, mit Diagrammen

Das Oracle fragen

Frageformen, die funktionieren

Wissen beibringen

Regeln schreiben, die bestehen

Tokens verwalten

Geltungsbereich, Rotation, Widerruf

Fehlersuche

Wenn es nicht hilft

Client-Referenz

Konfigurationsformen und Fallstricke je Tool